TP 1 — DevOps : ArtiShop

Réaliser par :

Sokaina Achbani Nouhaila El Bouhali Meryam Aznak Manar El Mahraz

Réponses aux questions sur Git et GitHub

Q1.1 Dans quel fichier les remotes sont-ils enregistrés ?

Dans le fichier .git/config du dépôt local. Ce fichier contient notamment les URL et les paramètres des remotes.

Q1.2 Pourquoi le push vers upstream est-il refusé ?

Parce que le dépôt original appartient à l'enseignante et que les étudiants n'ont normalement pas les droits d'écriture. Ils doivent pousser leurs modifications vers origin, leur fork commun.

Q2.1 Quel remote existe déjà dans votre clone, et pourquoi ? upstream y figure-t-il ?

- origin existe automatiquement, car il est configuré lors du clonage.
- upstream n'existe pas automatiquement : il faut l'ajouter manuellement avec git remote add upstream URL.
- Le graphe Git affiche les commits et les branches accessibles dans le dépôt local.

Chaque membre peut donc ajouter upstream s'il doit récupérer les mises à jour de l'enseignante.

Q3.1 À quelles deux commandes équivaut git pull ?

git fetch

git merge

Plus précisément, git pull récupère les changements de la branche distante suivie, puis les intègre dans la branche locale selon la stratégie configurée.

Q3.2 Pourquoi git fetch est-il qualifié de commande « sûre » ?

Parce qu'il récupère les nouveaux commits et met à jour les références distantes sans modifier directement la branche locale courante, les fichiers de travail ou les commits locaux. Il permet d'examiner les changements avant de les intégrer.

Q4.1 Quel risque court-on en commitant dans cet état ?

En état Detached HEAD, le commit créé n’est rattaché à aucune branche. Si l’on change ensuite de branche, ce commit risque de devenir difficile à retrouver et peut être perdu lors du nettoyage des objets Git.

Q5.1 Pourquoi Git n'a-t-il pas créé de commit de fusion, ni à l'étape 2 ni à l'étape 6 ?

Dans les deux étapes, si la branche à intégrer est un descendant direct de la branche de destination, Git réalise une fusion fast-forward. Il déplace simplement le pointeur de la branche vers un commit plus récent.

Il n'y a donc pas de nouveau commit de fusion. Si les historiques ont divergé, Git peut au contraire créer un commit de fusion

Q6.1 Quels sont les trois commits utilisés par cette fusion ?

Une fusion à trois voies s'appuie sur :

1. Le commit de base commun, appelé merge base.
2. Le dernier commit de la branche de destination main.
3. Le dernier commit de la branche source feature/livraison.

Pour vérifier le commit de base commun :

```
git merge-base main feature/livraison
```

Pour afficher les trois commits :

```
git log -1 --oneline $(git merge-base main feature/livraison)

git log -1 --oneline main

git log -1 --oneline feature/livraison
```

Q6.2 Membre 2 et Membre 3 ont tous deux modifié src/app.js. Pourquoi Git a-t-il pu fusionner sans conflit ?

Parce que Git compare les modifications par rapport à leur base commune. Si les changements sont situés dans des parties différentes du fichier, il peut les intégrer automatiquement. Modifier le même fichier ne signifie pas nécessairement modifier les mêmes lignes.

Q7.1 Pourquoi une fusion fast-forward ne produit-elle jamais de conflit ?

Une fusion fast-forward ne combine pas deux historiques divergents : elle avance simplement le pointeur de la branche vers un commit descendant. Git n'a donc pas besoin de comparer et de fusionner deux séries de modifications concurrentes.

Q9.1 Qu'aurait donné git pull --no-rebase ? Q9.2 Quelle est la « règle d'or » du rebase ?

La commande aurait récupéré les changements distants, puis tenté une fusion (merge) avec la branche locale. Comme les historiques ont divergé, elle aurait généralement créé un commit de fusion, au lieu de rejouer le commit local au-dessus du commit distant. Elle aurait aussi pu provoquer un conflit si les modifications étaient incompatibles.

Q9.2 — Quelle est la « règle d'or » du rebase ?

La règle d'or du rebase est de ne jamais effectuer un rebase sur une branche partagée ou déjà publiée sur laquelle d'autres collaborateurs travaillent, car cela réécrit l'historique des commits et peut provoquer des conflits pour les autres membres de l'équipe. Le rebase est à privilégier sur les branches locales personnelles.

Q11.1 Pourquoi utiliser --force-with-lease plutôt que --force ? Sur quelles branches ne force-t-on jamais ?

- **--force** remplace sans vérification suffisante la référence distante par l'historique local. Cela peut supprimer du travail publié par un autre collaborateur.
- **--force-with-lease** vérifie que la référence distante est toujours à la valeur attendue avant de la remplacer. Si quelqu'un a poussé de nouveaux commits entre-temps, l'opération est normalement refusée.

La sortie de git log --oneline --graph --all --decorate

```
PS C:\Users\hp\Desktop\ArtiShop> git log --oneline --graph --all --decorate
* 112e5f5 (origin/feature/avis) ajout des avis clients
| * ec07b53 (HEAD -> main, tag: v1.1.0, origin/main, origin/HEAD) style: changement couleur principale
|/
* d9a9b06 docs: correction du README
*   efdf28f merge: resolution du conflit
|\
| * 7444058 fix(promo): taux annonce par marketing
* | b43cf6d docs(promo): taux valide par la PO
|/
*   fe9d35d Merge branch 'feature/livraison'
|\
| * 2928565 feat(livraison): frais de livraison par ville
| * 5333466 (upstream/feature/livraison) feat(livraison): frais de livraison par ville
* | 83ad8ac feat(promo): codes promo BIENVENUE10 et RAMADAN20
* | 49dac5d docs: ajout
* | c923300 (upstream/main) ReadMe Update
|/
*   c0e265d (upstream/feature/promo) Fusion avec le dépôt GitHub initial
|\
| * 865adb5 Initial commit
* add0021 premier commit Artishop
```